



مدیریت منابع ابری و زمان بندی هسته های ناهمگن

در این پروژه، ما مفاهیم کلاسیک و مدرن سیستم عامل را در قالب یک سناریوی مرکز داده ابری پیاده سازی می کنیم. هدف اصلی ما درک عمیق دو چالش اساسی در سیستم عامل های امروزی است:

۱. همگام سازی (Synchronization): مدیریت دسترسی همزمان پردازش ها به منابع اشتراکی و جلوگیری از شرایط رقابت (Race Condition) و بن بست (Deadlock).
۲. زمان بندی ناهمگن (Asymmetric Scheduling): عبور از مدل های ساده زمان بندی و درک نحوه تخصیص وظایف در پردازنده های هیبریدی جدید (دارای هسته های ضعیف و قوی).

تا همین چند سال پیش، معماری پردازنده ها بر اساس پردازش متقارن بود؛ یعنی تمام هسته های یک CPU کاملاً شبیه به هم و دارای قدرت یکسانی بودند. زمان بند سیستم عامل (OS Scheduler) برای تخصیص یک پردازش یا ترد، نیازی نداشت نگران این باشد که کدام هسته را انتخاب می کند، زیرا همه هسته ها یکسان بودند.

اما با معرفی معماری big.LITTLE توسط ARM (در گوشی های هوشمند) و سپس ورود معماری ترکیبی (Hybrid) اینتل در پردازنده های نسل ۱۲ (Alder Lake)، موضوع پیچیده تر شد. اینتل دو نوع هسته کاملاً متفاوت را روی یک تراشه قرار داد:

- هسته های Performance: هسته های قدرتمند، فرکانس بالا، پرمصرف و مناسب برای پردازش های سنگین.
- هسته های Efficiency: هسته های کوچک تر، کم مصرف و مناسب برای پردازش های پس زمینه.

زمانی که این پردازنده ها وارد بازار شدند، یک مشکل بزرگ به وجود آمد. زمان بند سیستم عامل هایی مثل ویندوز ۱۰ برای این معماری طراحی نشده بودند و عملاً در برابر این

هسته‌ها کور بودند. سیستم‌عامل تفاوت بین هسته P و E را متوجه نمی‌شد. این موضوع باعث افت عملکرد سیستم می‌شد:

- گاهی سیستم‌عامل یک کارهای سنگین را به دلیل رفتار پس‌زمینه‌ای آن، روی هسته‌های ضعیف E قرار می‌داد که باعث افت شدید سرعت می‌شد.
- از طرف دیگر، کارهای ساده‌ای را روی هسته‌های قدرتمند P می‌انداخت که باعث مصرف شدید باتری و داغ شدن بی‌مورد سیستم می‌شد.

این‌تل مجبور شد یک قطعه سخت‌افزاری به نام Intel Thread Director بسازد تا رفتار تردها را مانیتور کند و به زمان‌بند ویندوز ۱۱ تقلب برساند که هر ترد چه ویژگی‌هایی دارد تا سیستم‌عامل بتواند تردهای سنگین را به P-Cores و تردهای سبک را به E-Cores منتقل کند. (برای مطالعه بیشتر درباره عملکرد Thread Director می‌توانید به این [لینک](#) مراجعه کنید).

شما در این پروژه وظیفه دارید بخش‌هایی از یک سیستم‌عامل را برای یک سرور ابری مدرن بنویسید. این پروژه در دو فاز اصلی انجام می‌شود:

- فاز اول (مدیریت منابع و همگام‌سازی): سرور شما درخواست‌های زیادی از کاربران دریافت می‌کند که باید همزمان پردازش شوند. شما باید با استفاده، Mutex منابع سرور (مثل دیتابیس‌ها) را مدیریت کنید تا اطلاعات خراب نشوند (به دلیل Race Condition) و سیستم در اثر تخصیص نادرست منابع قفل نکند (Deadlock).
- فاز دوم (زمان‌بند ناهمگن): در این فاز، شما زمان‌بند (Scheduler) سرور را می‌نویسید. سرور شما دارای هسته‌های قدرتمند (P) و هسته‌های اقتصادی (E) است. شما باید الگوریتمی پیاده‌سازی کنید که درخواست‌های حیاتی و سنگین را به هسته‌های P و کارهای پس‌زمینه را به هسته‌های E اختصاص دهد.

فاز اول: مدیریت منابع اشتراکی، همگام‌سازی و بن‌بست

برای انجام این فاز، شما باید یک سیستم ساده‌ی مدیریت حساب‌های کاربری طراحی کنید. در کد خود به یک لیست از حساب‌ها نیاز دارید. از یک کلاس (یا استراکت) به نام Account استفاده کنید که دارای فیلدهای زیر باشد:

- int account_id (شناسه‌ی یکتای کاربر)
- double balance (موجودی حساب)
- pthread_mutex_t lock (قفل اختصاصی برای هر حساب)

همچنین به یک متغیر سراسری به نام Total_Revenue (درآمد کل سرور) نیاز دارید.

شرایط رقابت (Race Condition) و رفع آن (25 نمره)

هر بار که کاربران فایلی در سرور آپلود می‌کنند، مبلغی به عنوان کارمزد به درآمد کل سرور (Total_Revenue) اضافه می‌شود. می‌خواهیم رفتار همزمان هزاران درخواست آپلود را شبیه‌سازی کنیم.

- بخش اول: برنامه‌ای بنویسید که N ترد ایجاد کند. هر ترد نشان‌دهنده‌ی یک درگاه پردازشی است که در یک حلقه، K بار مقدار ۱۰۰ را به Total_Revenue اضافه می‌کند. در این بخش از هیچ قفلی استفاده نکنید.
- بخش دوم: یک pthread_mutex_t به نام revenue_lock تعریف کنید و عملیات جمع را در یک بخش بحرانی (Critical Section) قرار دهید تا مشکل همگام‌سازی حل شود.

قالب ورودی و خروجی خاصی مد نظر نیست ولی حتما باید مقداری که انتظار داشتیم Total_Revenue میشد و مقداری که در هر بخش در عمل بدست میاد را چاپ کنید تا بتوان مقایسه و تحلیل کرد.

پارامترهای مانند اینکه چند ترد داشته باشیم و هر کدام چند بار جمع بزنند را تنظیم کنید. اگر تعداد جمع زدن‌ها به ازای هر ترد خیلی کم باشد یا خیلی زیاد باشد چه تفاوتی در خروجی این قسمت می‌بینید؟ چرا؟

شبیه‌سازی و رفع بن‌بست (25 نمره)

کاربران سرور ابری می‌توانند اعتبار مالی خود را به حساب یکدیگر انتقال دهند. برای انتقال وجه از حساب A به حساب B، سرور باید همزمان قفل هر دو حساب را در اختیار بگیرد تا در حین انتقال، هیچ ترد دیگری نتواند موجودی آن‌ها را تغییر دهد.

- بخش اول: دو ترد به نام‌های Thread_1 و Thread_2 بسازید.
 - Thread_1 قصد دارد مبلغی را از Account_0 به Account_1 منتقل کند (ابتدا Account_0 را قفل می‌کند، یک sleep کوتاه مثلاً ۱۰۰ میلی‌ثانیه انجام می‌دهد و سپس تلاش می‌کند Account_1 را قفل کند).
 - Thread_2 همزمان قصد دارد مبلغی را از Account_1 به Account_0 منتقل کند (ابتدا Account_1 را قفل می‌کند، sleep می‌کند و سپس تلاش می‌کند Account_0 را قفل کند).
 - در این حالت سیستم باید کاملاً قفل شود.
- بخش دوم:

برای حل این مشکل از استراتژی ترتیب‌دهی منابع (Resource Ordering) استفاده کنید. قانونی در تابع انتقال وجه بنویسید که: همیشه حسابی که account_id کوچکتری دارد باید اول قفل شود، صرف نظر از اینکه فرستنده یا گیرنده است.

فاز دوم: زمان بندی ناهمگن (Asymmetric Scheduling)، صف های چند سطحی و مصرف انرژی

برای انجام این فاز، شما باید یک شبیه ساز زمان بندی طراحی کنید که پردازش ها را هوشمندانه روی این هسته های ناهمگن پخش کرده و مصرف انرژی سیستم را مدیریت کند.

ساختار پردازش: در کد خود، به یک کلاس (یا استراکت) به نام Process نیاز دارید که حداقل دارای فیلدهای زیر باشد:

- int pid (شناسه ی یکتای پروسه)
- int arrival_time (زمان ورود به سیستم)
- int burst_time (کل زمان نیاز به پردازنده)
- int e_core_ticks و int p_core_ticks (شمارنده هایی برای ذخیره تعداد تیک هایی که پروسه روی هر نوع هسته اجرا شده است جهت محاسبه مصرف انرژی نهایی)
- و فیلدهای لازم برای رهگیری وضعیت (زمان باقیمانده ی پردازش، زمان انتظار، زمان پاسخ دهی، اولویت فعلی صف و...).

(20 نمره) بخش اول: شبیه سازی صف های چند سطحی با بازخورد (MLFQ)

- **ساختار صف ها:** سه صف آماده (Ready Queue) با اولویت های مختلف بسازید:
 - **صف اول (Q1 - بالاترین اولویت):** با الگوریتم Round Robin و کوانتوم ۲.
 - **صف دوم (Q2 - اولویت متوسط):** با الگوریتم Round Robin و کوانتوم ۴.
 - **صف سوم (Q3 - پایین ترین اولویت):** با الگوریتم FCFS.
- **تغییر اولویت ها:**
 - **تنزل (Demotion):** تمامی پروسه های جدید در ابتدا وارد Q1 می شوند. اگر پروسه ای در Q1 یا Q2 تمام کوانتوم خود را روی پردازنده مصرف کرد اما پردازش آن تمام نشد، به این معناست که یک پردازش نسبتاً سنگین است و باید متوقف شده و به انتهای صف سطح پایین تر سقوط کند.
 - **ارتقا (Aging):** برای جلوگیری از گرسنگی (Starvation) پروسه های محاسباتی، قانونی پیاده سازی کنید که اگر پروسه ای در صف های پایین (Q2 یا Q3) به مدت متوالی ۱۰ تیک زمانی در صف منتظر ماند، فوراً به انتهای Q1 ارتقا یابد.

○ **پیش‌دستی (Preemption):** در این سیستم، اولویت صف Q1 مطلق است و هسته‌ی قدرتمند P-Core باید همواره در خدمت این صف باشد. اگر در یک تیک زمانی، پروسه‌ای به Q1 اضافه شود (چه پروسه تازه‌وارد و چه پروسه‌ای که به دلیل Aging ارتقا یافته است)، اما هسته‌ی P-Core در حال اجرای پروسه‌ای از صف Q2 باشد، سیستم‌عامل باید بلافاصله پروسه Q2 را متوقف (Preempt) کند. در این حالت، پروسه‌ی متوقف‌شده نباید تنزل اولویت پیدا کند؛ بلکه باید به ابتدای همان صف Q2 بازگردد تا نوبت خود را از دست ندهد. سپس هسته‌ی P-Core بی‌درنگ در اختیار پروسه‌ی منتظر در Q1 قرار می‌گیرد. (توجه: پروسه‌های حاضر در Q3 که روی E-Core اجرا می‌شوند، توسط Q1 پیش‌دستی نمی‌شوند، زیرا Q1 کاری با هسته‌های E-Core ندارد).

ترتیب دقیق اجرای رویدادها در هر تیک زمانی (Tick): از آنجا که هسته‌ها به صورت همزمان (Concurrent) کار می‌کنند، برای جلوگیری از تداخل و ابهام در پیاده‌سازی، حلقه اصلی شبیه‌ساز شما در هر تیک زمانی باید مراحل زیر را دقیقاً به همین ترتیب اجرا کند:

1. **ورود (Arrival):** ابتدا بررسی کنید آیا پروسه جدیدی در این تیک وارد سیستم شده است؟ در صورت وجود، آن را به انتهای صف Q1 اضافه کنید.
2. **ارتقا (Aging) و زمان انتظار:** زمان انتظار تمامی پروسه‌های منتظر در هر سه صف را یک واحد افزایش دهید. همچنین به عنوان مکانیزم Aging، بررسی کنید اگر پروسه‌ای در صف‌های Q2 یا Q3 به مدت متوالی ۱۰ تیک منتظر مانده است، فوراً آن را از صف فعلی خارج کرده و به انتهای Q1 منتقل کنید.
3. **پیش‌دستی (Preemption):** بررسی کنید آیا پروسه‌ای در Q1 وجود دارد که منتظر P-Core باشد، در حالی که P-Core توسط پروسه‌ای از Q2 اشغال شده است؟ اگر بله، عملیات پیش‌دستی را انجام دهید (توضیحات پیش‌دستی در ادامه آمده است).
4. **تخصیص هسته‌های آزاد (Allocation):** اکنون هسته‌هایی که بی‌کار (Idle) هستند، بر اساس قوانین مجاز خود، پروسه جدیدی را از سر صف‌ها برمی‌دارند.
5. **اجرای همزمان (Execution):** تمام هسته‌های درگیر، کار خود را به صورت موازی انجام می‌دهند (کاهش ۲ واحد پردازش برای P-Core و ۱ واحد برای E-Core). کوانتوم مصرفی پروسه‌ها در این مرحله یک واحد افزایش می‌یابد.

6. **خاتمه یا تنزل (Termination / Demotion):** در پایان تیک بررسی کنید: آیا نیاز پردازشی پروسه‌ای صفر شده است؟ (خاتمه). اگر نه، آیا کوانتوم مجاز آن در صف فعلی پر شده است؟ اگر بله، آن را از هسته خارج کرده و به انتهای صف پایین‌تر (تنزل) بفرستید.

(15 نمره) بخش دوم: زمان‌بندی ناهمگن (Heterogeneous SMP) و

مدیریت انرژی

شبیه‌ساز ما دارای ۳ هسته است: ۱ هسته‌ی (P-Core Performance) و ۲ هسته‌ی (E-Core Efficiency).

سرعت و مصرف انرژی:

- **هسته‌ی P-Core:** بسیار سریع اما پرمصرف است. اجرای یک پروسه روی این هسته، در هر تیک زمانی سیستم ۲ واحد از زمان پردازش باقیمانده‌ی پروسه کم می‌کند، اما ۵ وات انرژی در آن تیک مصرف می‌کند.
- **هسته‌های E-Core:** کندتر اما بهینه هستند. در هر تیک زمانی ۱ واحد پردازش انجام می‌دهند و تنها ۱ وات انرژی در آن تیک مصرف می‌کنند.
- **فرمول محاسبه انرژی مصرفی کل:**
برای محاسبه انرژی مصرفی هر پروسه و در نهایت کل سیستم، از فرمول زیر استفاده می‌شود:
$$\text{Total Energy} = (\text{P-Core Ticks} * 5) + (\text{E-Core Ticks} * 1)$$

(راهنما: P-Core Ticks مجموع تیک‌هایی است که پروسه روی هسته P-Core اجرا شده و E-Core Ticks مجموع تیک‌هایی است که روی هسته‌های E-Core اجرا شده است.)
- **تخصیص هوشمند منابع (Resource Allocation):** زمان‌بند شما باید بر اساس صف‌ها تصمیم‌گیری کند:
 - پروسه‌های حاضر در Q1 باید فقط روی P-Core قرار گیرند تا در سریع‌ترین زمان ممکن پاسخ داده شوند.

○ پروسه‌های حاضر در Q3 (پروسه‌های سنگین پس‌زمینه که کوانتوم‌های خود را سوزانده‌اند) فقط باید روی E-Cores زمان‌بندی شوند تا از داغ شدن سیستم و هدررفت انرژی جلوگیری شود.

○ پروسه‌های حاضر در Q2 می‌توانند روی هر هسته‌ای که آزاد است (P یا E) قرار بگیرند.

سناریوهای تستی : برای ارزیابی صحت عملکرد شبیه‌ساز، باید ۳ سناریوی زیر را در برنامه خود تعریف (Hard-code) کرده و خروجی‌های خواسته شده را برای هرکدام چاپ و تحلیل کنند:

سناریوی اول: پایه و تعاملی (بررسی تنزل اولویت و تخصیص اولیه)

این سناریو برای اطمینان از عملکرد درست صف Q1، انتقال به Q2 و تخصیص صحیح هسته P-Core به پروسه‌های تازه‌وارد طراحی شده است.

نیاز پردازشی (Burst Time)	زمان ورود (Arrival Time)	پردازه (PID)
10	0	P1
12	0	P2
2	1	P3

سناریوی دوم: محاسبات سنگین و گرسنگی (بررسی مکانیزم Aging)

در این سناریو، پروسه‌های سنگین به سرعت به Q3 سقوط کرده و روی E-Cores قرار می‌گیرند. این سناریو مشخصاً برای تست مکانیزم Aging (ارتقا پس از ۱۰ تیک انتظار) طراحی شده است.

نیاز پردازشی (Burst Time)	زمان ورود (Arrival Time)	پردازه (PID)
20	0	P1
25	2	P2
5	5	P3

سناریوی سوم: ورودهای متناوب (بررسی Preemption و Idle شدن هسته‌ها)

این سناریو دارای زمان‌های ورود با فواصل زیاد است. زمان‌بند باید بتواند بی‌کار شدن هسته‌ها (Idle) و همچنین پیش‌دستی (Preemption) را در زمان ورود پروسه‌های جدید (مانند P4) به درستی مدیریت کند.

نیاز پردازشی (Burst Time)	زمان ورود (Arrival Time)	پردازه (PID)
8	0	P1
15	3	P2
6	10	P3
3	12	P4

(15 نمره) بخش سوم: خروجی و تحلیل

قالب دریافت ورودی‌ها آزاد است و می‌توانید سناریوهای تستی را در تابع main به صورت هاردکد تعریف کنید. برنامه شما در پایان شبیه‌سازی باید خروجی‌های زیر را در ترمینال چاپ کند:

۱. **لاگ اجرایی (Execution Gantt Chart):** یک گزارش متنی که نشان دهد در هر تیک زمانی، کدام پروسه روی کدام هسته در حال اجراست و آیا هسته‌ای بی‌کار (Idle) مانده است یا خیر.

۲. **جدول آمار:** به ازای هر پروسه مقادیر زمان بازگشت (Turnaround Time)، زمان انتظار (Waiting Time) و زمان پاسخ‌دهی (Response Time) را چاپ کنید. در نهایت میانگین این زمان‌ها، درصد بهره‌وری کل پردازنده‌ها (CPU Utilization) و کل انرژی مصرفی سیستم (بر حسب وات) را نمایش دهید.

(راهنما: برای محاسبه‌ی دقیق و راحت‌تر زمان انتظار کل (Waiting Time) برای هر پروسه، می‌توانید به جای شمارش دستی، در زمان خاتمه‌ی پروسه از فرمول قطعی زیر استفاده کنید:

$$\text{Waiting Time} = \text{Turnaround Time} - (\text{P-Core Ticks} + \text{E-Core Ticks})$$

سوالات تحلیلی :

- پارامترهایی مانند زمان رسیدن پروسه‌ها و طول پردازش آن‌ها را تغییر دهید. اگر مکانیزم MLFQ را به طور کامل حذف کنید و تمام پروسه‌ها را صرفاً بر روی P-Core اجرا کنید، چه تفاوتی در خروجی انرژی مصرفی کل و زمان پاسخ‌دهی می‌بینید؟ چرا؟
- با توجه به اینکه در این فاز عملیات I/O شبیه‌سازی نمی‌شود، نقش مکانیزم Aging در درگیر نگه داشتن هسته‌ی P-Core چیست؟ اگر Aging را غیرفعال کنید، چه اتفاقی برای هسته‌ی P-Core و زمان اتمام کل شبیه‌سازی می‌افتد؟

نکات و نحوه تحویل

- کدهای شما می بایست به زبان C++/C بوده و با استفاده از Makefile و کامپایلر g++ قابل کامپایل و اجرا باشند.
 - پروژه شما میبایست در سیستم عامل Linux و در زمان معقول قابل اجرا باشد.
 - تمامی کدها و فایل‌های خود را به صورت zip شده و به فرمت OS-CA3-<SID>.zip در صفحه درس بارگذاری کنید.
 - در صورت داشتن سوال میتوانید از طریق فروم یا گروه درس و یا شرکت در جلسات رفع اشکال سوالات خود را مطرح کنید.
 - تمامی مواردی که در فروم یا گروه درس و یا جلسات رفع اشکال به آن اشاره می شود جزئی از این تمرین خواهند بود.
 - این تمرین صرفاً برای یادگیری شما طراحی شده است. در صورت محرز شدن هرگونه تخلف، مطابق قوانین درس با آن برخورد خواهد شد.
- موفق باشید.